

Chapter -2: UML Diagram

- **An overview of UML**
- **Building Blocks of the UML**
- **Relationships in the UML**
- **Diagrams in the UML (Use Case Diagrams, Class Diagrams, Sequence diagrams, State chart diagrams, Activity diagrams, Component diagram, Deployment diagram)**

2.1 An overview of UML

- Unified Modeling Language (UML) is a general-purpose modeling language.
- The main aim of UML is to define a standard way to visualize the way a system has been designed.
- UML is not a programming language, it is rather a visual language.

overview of UML(cont'd)

- Software system artifacts can be specified, visualized, built, and documented with the use of UML.
 - ✓ We use UML diagrams to show the behavior and structure of a system.
 - ✓ UML helps software engineers, businessmen, and system architects with modeling, design, and analysis.

overview of UML(cont'd)

Why do we need UML?

- Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate amongst them.
- Businessmen do not understand code. So UML becomes essential to communicate with non-programmers about essential requirements, functionalities, and processes of the system.
- A lot of time is saved down the line when teams can visualize processes, user interactions, and the static structure of the system.

(cont'd)

- The first thing to notice about the UML is that there are a lot of different diagrams (models) to get used to. The reason for this is that it is possible to look at a system from many different viewpoints. A software development will have many stakeholders playing a part.
- For Example:
 - ✓ Analysts
 - ✓ Designers
 - ✓ Coders
 - ✓ Testers
 - ✓ QA
 - ✓ The Customer
 - ✓ Technical Authors

(cont'd)

- All of these people are interested in different aspects of the system, and each of them require a different level of detail.
- For example, a coder needs to understand the design of the system and be able to convert the design to a low level code.
- By contrast, a technical writer is interested in the behavior of the system as a whole, and needs to understand how the product functions.
- The UML attempts to provide a language so expressive that all stakeholders can benefit from at least one UML diagram.

2.2 A Conceptual Model of UML

A Conceptual Model of UML

To understand the conceptual model of UML, first we need to clarify what is a conceptual model? and why a conceptual model is required?

- A conceptual model can be defined as a model which is made of concepts and their relationships.
- A conceptual model is the first step before drawing a UML diagram. It helps to understand the entities in the real world and how they interact with each other.
- The conceptual model of UML can be mastered by learning the following three major elements
 - ✓ UML building blocks
 - ✓ Rules to connect the building blocks
 - ✓ Common mechanisms of UML

2.2.1 Building Blocks of the UML

- The building blocks of UML can be defined as
 - Things

Things are the most important building blocks of UML. Things can be

 - ✓ Structural
 - ✓ Behavioral
 - ✓ Grouping
 - ✓ Annotational
 - Relationships
 - ✓ Dependency
 - ✓ Association
 - ✓ Generalization
 - ✓ Realization
 - Diagrams
 - ✓ Structure diagrams
 - ✓ Behaviors diagrams

2.2.1.1 Things

- **Structural Things**

define the static part of the model. They represent the physical and conceptual elements. Following are the brief descriptions of the structural things.

- **Class** – Class represents  of objects having similar responsibilities.

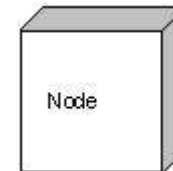
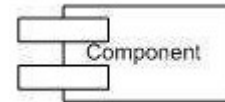
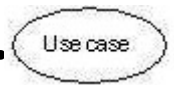
- **Interface** – Interface defines a set of operations which specify the responsibility of a class.



- **Collaboration** – Collaboration defines an interaction between elements.

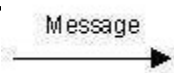
Structural Things(cont'd)

- **Use case** –Use case represents a set of actions performed by a system for a specific goal.
- **Component** –Component describes the physical part of a system.
- **Node** – A node can be defined as a physical element that exists at run time.



2.2.1.1 Things

Behavioral Things

- **A behavioral thing** consists of the dynamic parts of UML models. Following are the behavioral things –
- **Interaction** – Interaction is defined as a behavior that consists of a group of messages exchanged among elements to accomplish a specific task. 
- **State machine** – State machine is useful when the state of an object in its life cycle is important. It defines the sequence of states an object goes through in response to events. Events are external factors responsible for state change. 

2.2.1.1 Things

Grouping Things

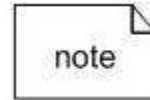
- **Grouping things** can be defined as a mechanism to group elements of a UML model together. There is only one grouping thing available.
- **Package** – Package is the only one grouping thing available for gathering structural and behavioral things.



2.2.1.1 Things

Annotational Things

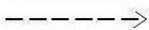
- **Annotational things** can be defined as a mechanism to capture remarks, descriptions, and comments of UML model elements.
- **Note** - It is the only one Annotational thing available. A note is used to render comments, constraints, etc. of an UML element.



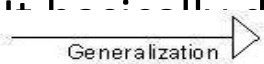
2.2.1.2 Relationship

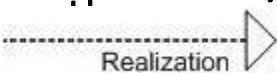
Relationship is another most important building block of UML. It shows how the elements are associated with each other and this association describes the functionality of an application.

- There are four kinds of relationships available.

Dependency is a relationship between two things in which change in one element also affects the other. 

Association is basically a set of links that connects the elements of a UML model. It also describes how many objects are taking part in that relationship. 

Generalization can be defined as a relationship which connects a specialized element with a generalized element.  describes the inheritance relationship in the world of objects.

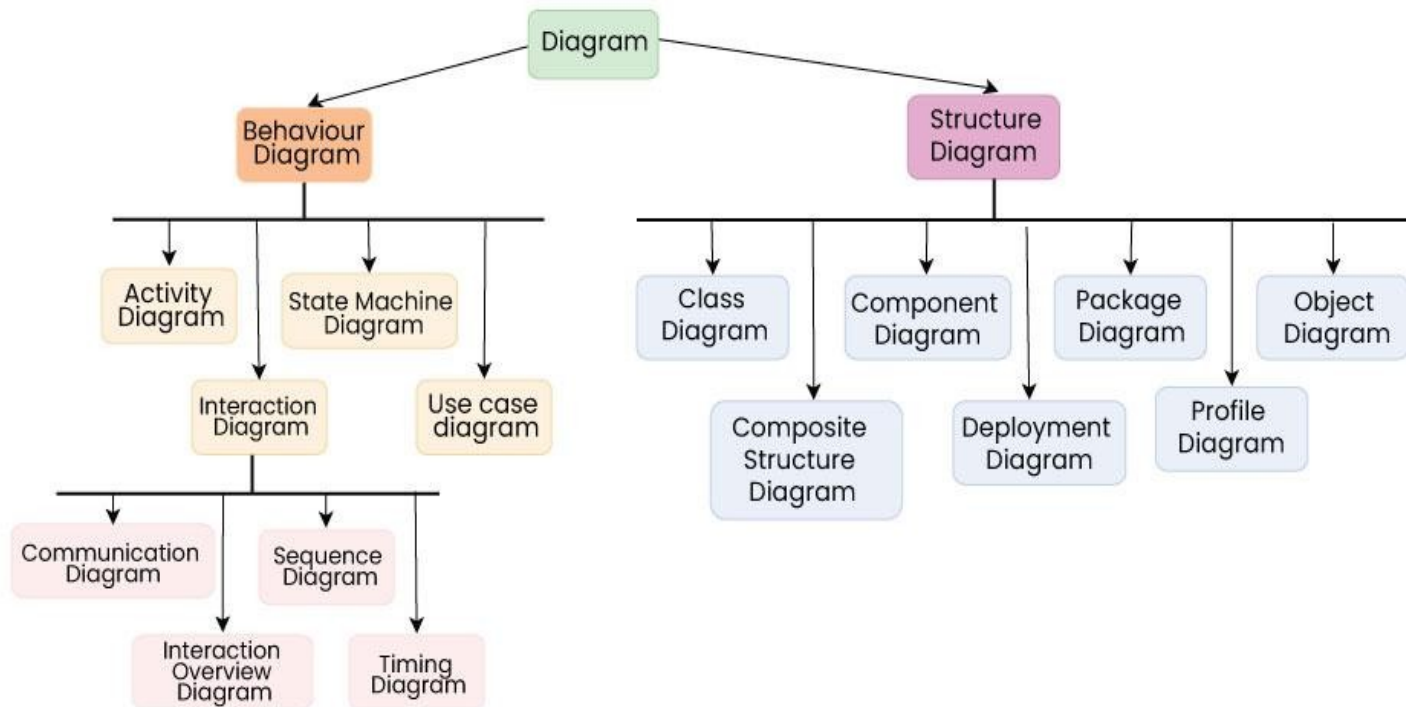
Realization can be defined as a relationship in which two elements are connected. One element describes some responsibility, which is not implemented and the other implements them. This relationship exists in case of interfaces. 

2.2.1.3 UML Diagrams

- UML diagrams are the ultimate output of the entire discussion.
- All the elements, relationships are used to make a complete UML diagram and the diagram represents a system.
- The visual effect of the UML diagram is the most important part of the entire process. All the other elements are used to make it complete.

UML Diagrams(cont'd)

- Types of UML



UML Diagrams(cont'd)

- Here's a quick look at each one of these 13 diagrams in as shown in the UML 2 Diagram Structure below:
- Structure diagrams show the static structure of the system and its parts on different abstraction and implementation levels and how they are related to each other.
- The elements in a structure diagram represent the meaningful concepts of a system, and may include abstract, real world and implementation concepts, there are seven types of structure diagram as follows:
 - ✓ **Class Diagram**
 - ✓ **Component Diagram**
 - ✓ **Deployment Diagram**
 - ✓ **Object Diagram**
 - ✓ **Package Diagram**
 - ✓ **Composite Structure Diagram**
 - ✓ **Profile Diagram**

UML Diagrams(cont'd)

- Behavior diagrams show the **dynamic behavior** of the objects in a system, which can be described as a series of changes to the system over **time**, there are seven types of behavior diagrams as follows:
 - ✓ Use Case Diagram
 - ✓ Activity Diagram
 - ✓ State Machine Diagram
 - ✓ Sequence Diagram
 - ✓ Communication Diagram
 - ✓ Interaction Overview Diagram
 - ✓ Timing Diagram

An overview of UML

- Class Diagram

The class diagram is a central modeling technique that runs through nearly all object-oriented methods.

This diagram describes the types of objects in the system and various kinds of static relationships which exist between them.

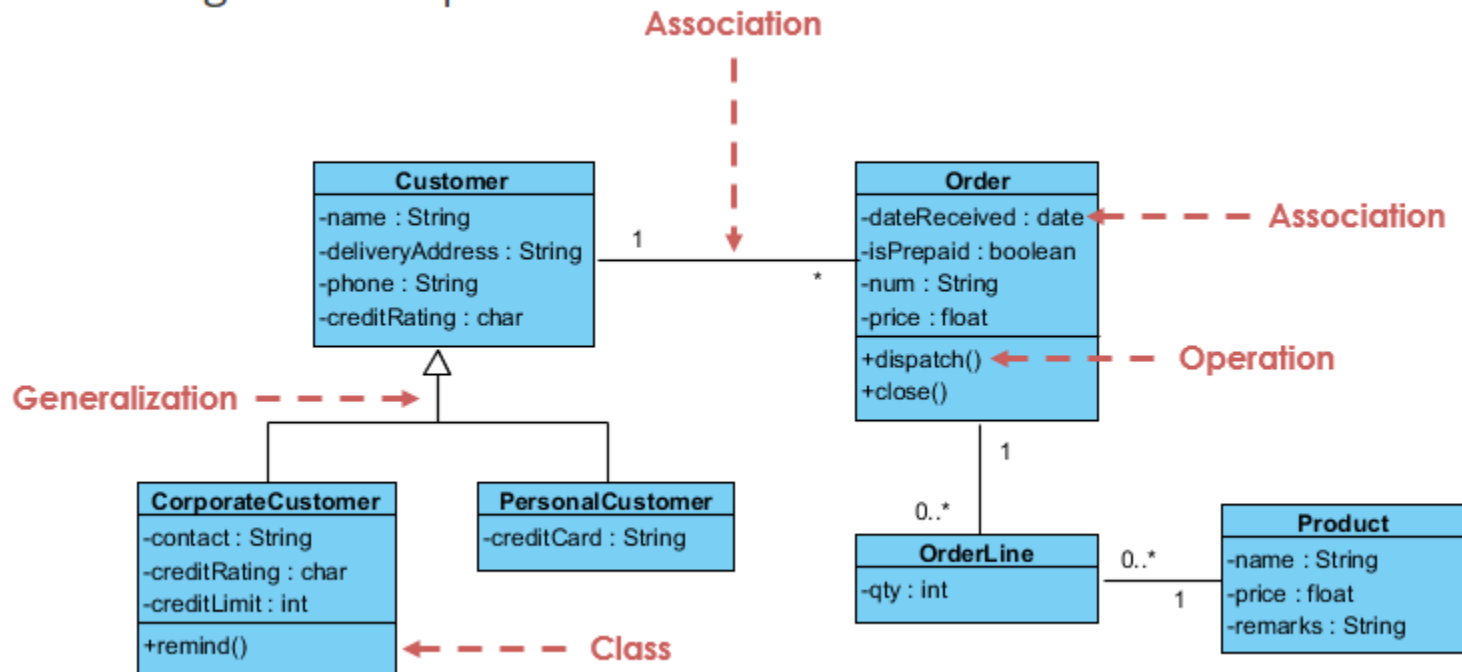
Relationships

There are three principal kinds of relationships which are important:

- **Association** - represent relationships between instances of types (a person works for a company, a company has a number of offices).
- **Inheritance** - the most obvious addition to ER diagrams for use in OO. It has an immediate correspondence to inheritance in OO design.
- **Aggregation** - Aggregation, a form of object composition in object-oriented design.

Class Diagram

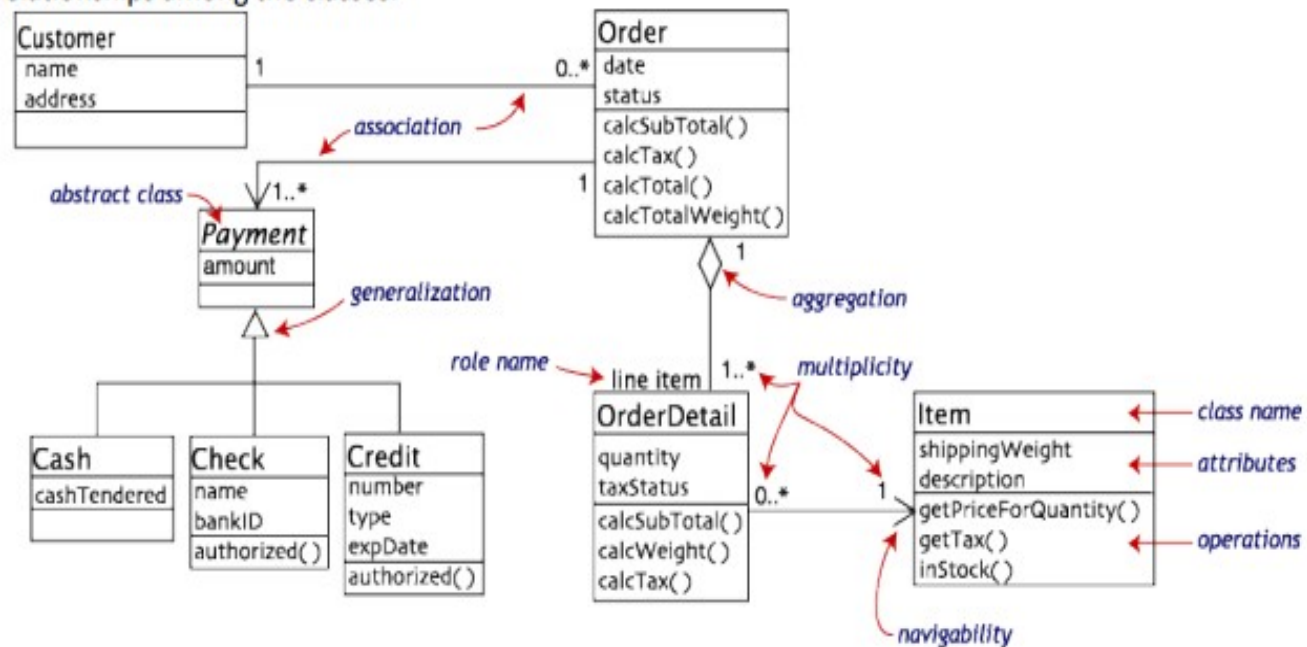
Class Diagram Example



Cont'd

1. Class Diagram

Describes the structure of a system by showing the system's classes, their attributes, and the relationships among the classes.

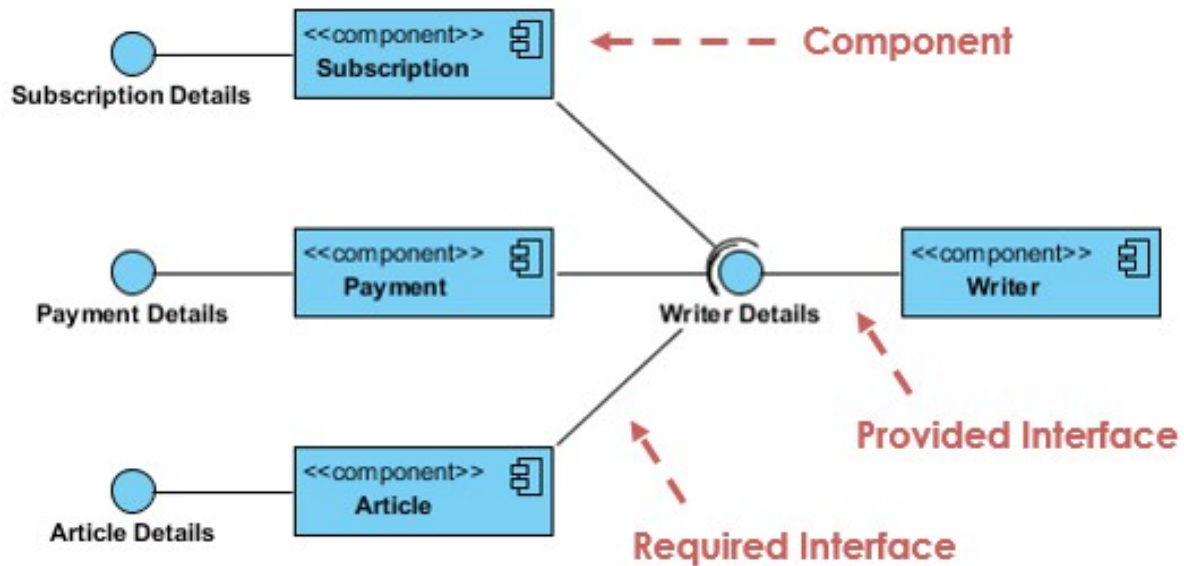


Component Diagram

- In the Unified Modeling Language, a component diagram depicts how components are wired together to form larger components or software systems.
- It illustrates the architectures of the software components and the dependencies between them.
- Those software components including run-time components, executable components also the source code components.

Component diagram

Component Diagram Example

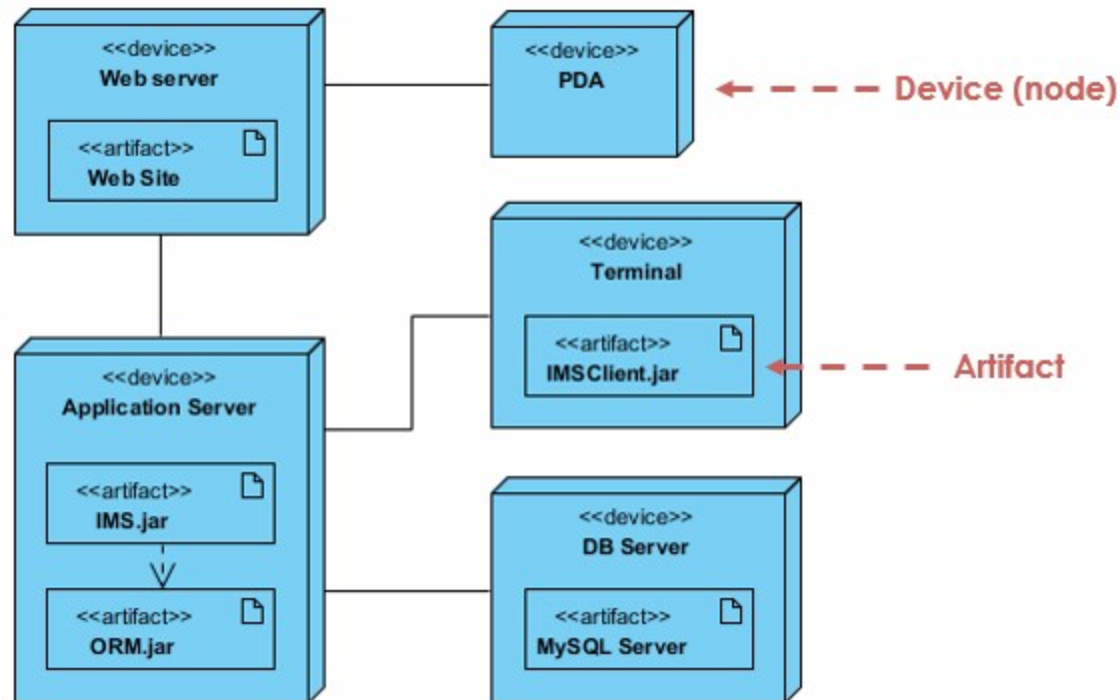


Deployment Diagram

- The Deployment Diagram helps to model the physical aspect of an Object-Oriented software system.
- It is a structure diagram which shows architecture of the system as deployment (distribution) of software artifacts to deployment targets.
- Artifacts represent concrete elements in the physical world that are the result of a development process.
- It models the run-time configuration in a static view and visualizes the distribution of artifacts in an application.
- In most cases, it involves modeling the hardware configurations together with the software components that lived on.

Deployment Diagram

Deployment Diagram Example



Object Diagram

- An object diagram is a graph of instances, including objects and data values.
- A static object diagram is an instance of a class diagram; it shows a snapshot of the detailed state of a system at a point in time.
- The difference is that a class diagram represents an abstract model consisting of classes and their relationships.
- However, an object diagram represents an instance at a particular moment, which is concrete in nature.

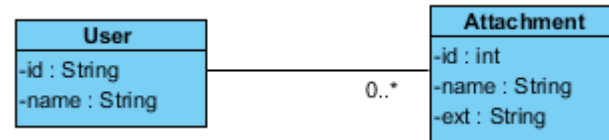
Relationship between Class Diagram and Object Diagram

- You create "classes" when you are programming. For example, in an online banking system you may create classes like 'User', 'Account', 'Transaction', etc.
- In a classroom management system you may create classes like 'Teacher', 'Student', 'Assignment', etc.
- In each class, there are attributes and operations that represent the characteristic and behavior of the class.
- Class Diagram is a UML diagram where you can visualize those classes, along with their attributes, operations and the inter-relationship.

Cont'd

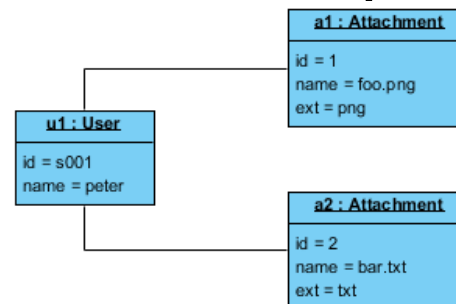
- UML Object Diagram shows how object instances in your system are interacting with each other at a particular state. It also represents the data values of those objects at that state.

Class Diagram Example



- In other words, a UML Object Diagram can be seen as a representation of how classes (drawn in UML Class Diagram) are utilized at a particular state.

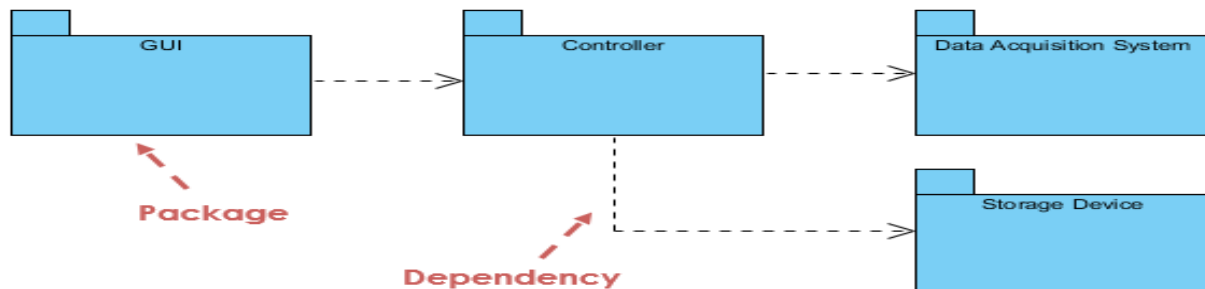
Object Diagram Example



Package Diagram

- Package diagram is UML structure diagram which shows packages and dependencies between the packages. Model diagrams allow to show different views of a system, for example, as multi-layered (aka multi-tiered) application - multi-layered application model.

Package Diagram Example

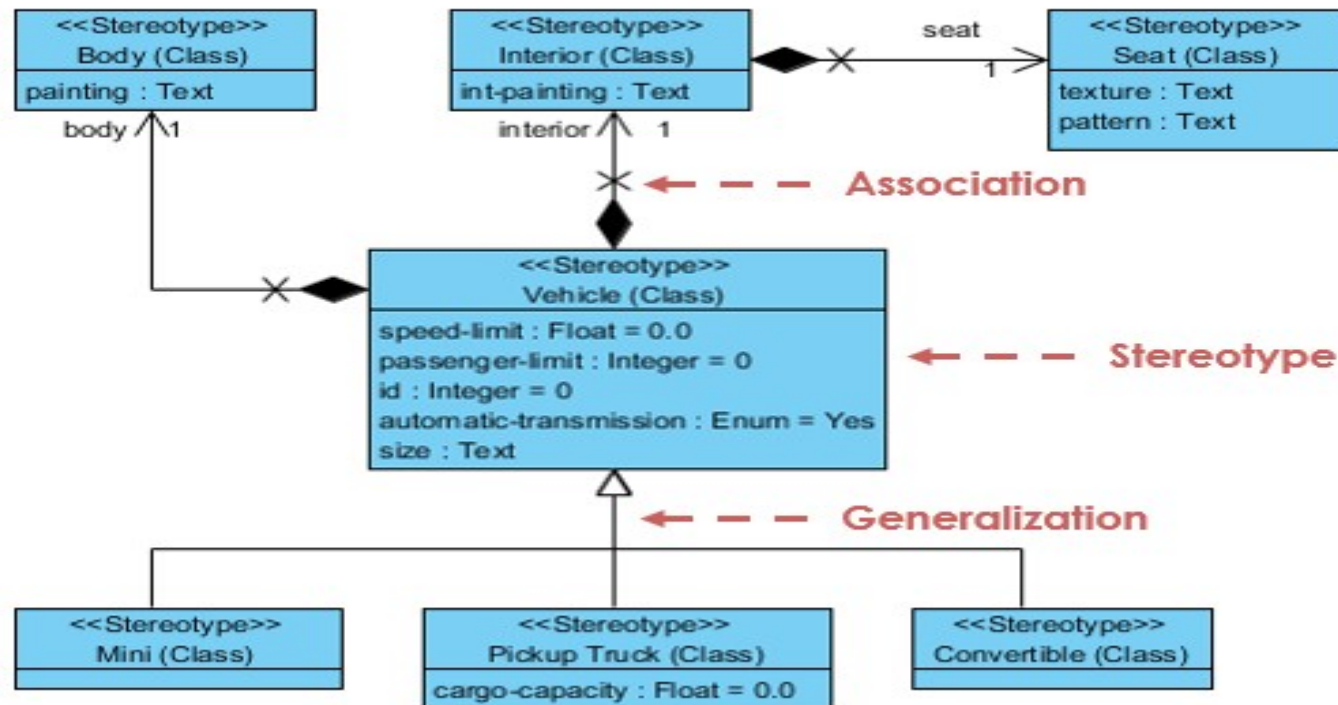


Profile Diagram

- A profile diagram enables you to create domain and platform specific stereotypes and define the relationships between them.
- You can create stereotypes by drawing stereotype shapes and relate them with composition or generalization through the resource-centric interface. You can also define and visualize tagged values of stereotypes.

Cont'd

Profile Diagram Example

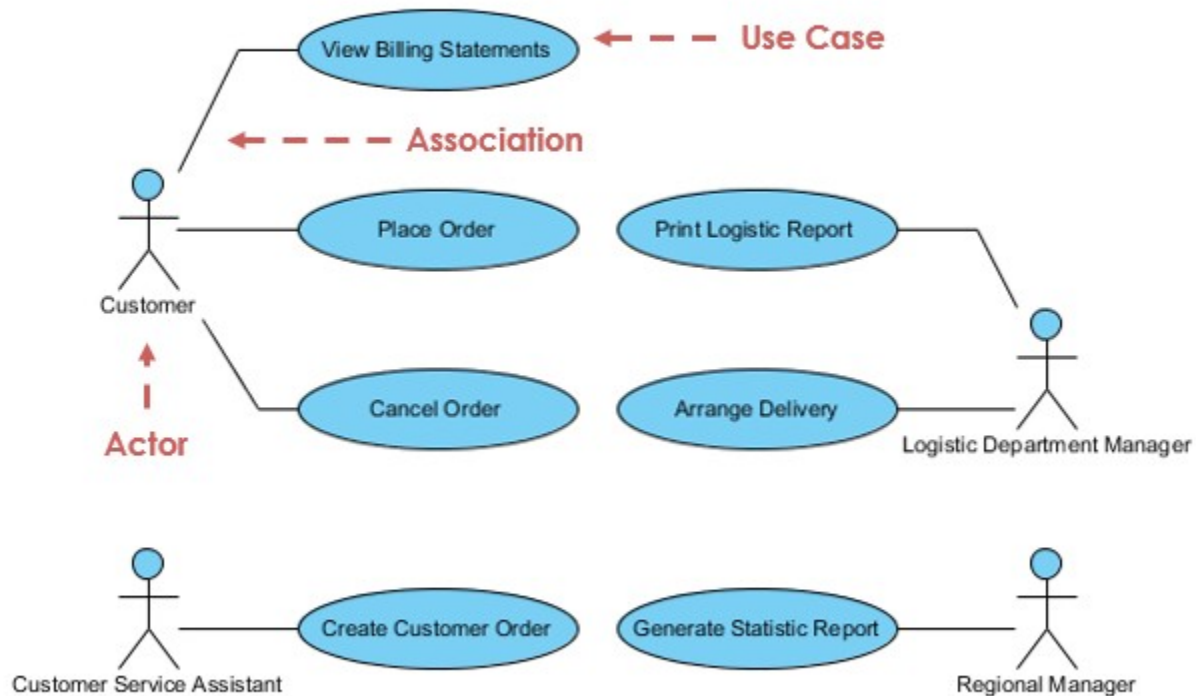


Use Case Diagram

- A use-case model describes a system's functional requirements in terms of use cases. It is a model of the system's intended functionality (use cases) and its environment (actors). Use cases enable you to relate what you need from a system to how the system delivers on those needs.
- *Think of a use-case model as a menu*, much like the menu you'd find in a restaurant. By looking at the menu, you know what's available to you, the individual dishes as well as their prices.
- By looking at the menu, you get an overall impression of the dining experience that awaits you in that restaurant.

Cont'd

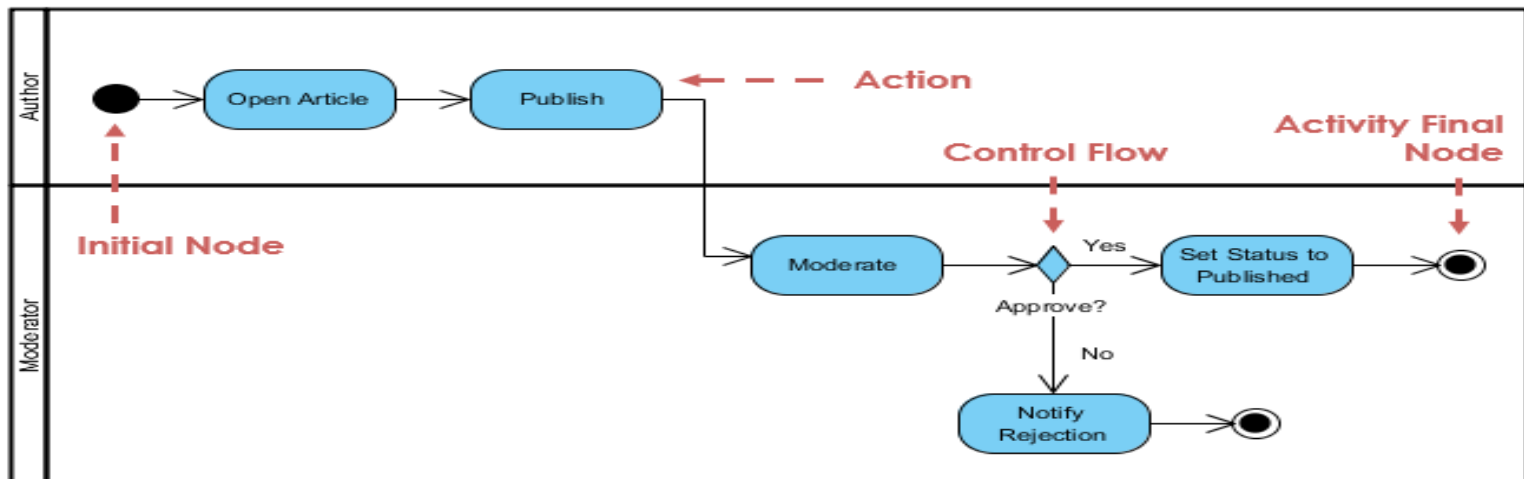
Use Case Diagram Example



Activity Diagram

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. It describes the flow of control of the target system, such as the exploring complex business rules and operations, describing the use case also the business process.

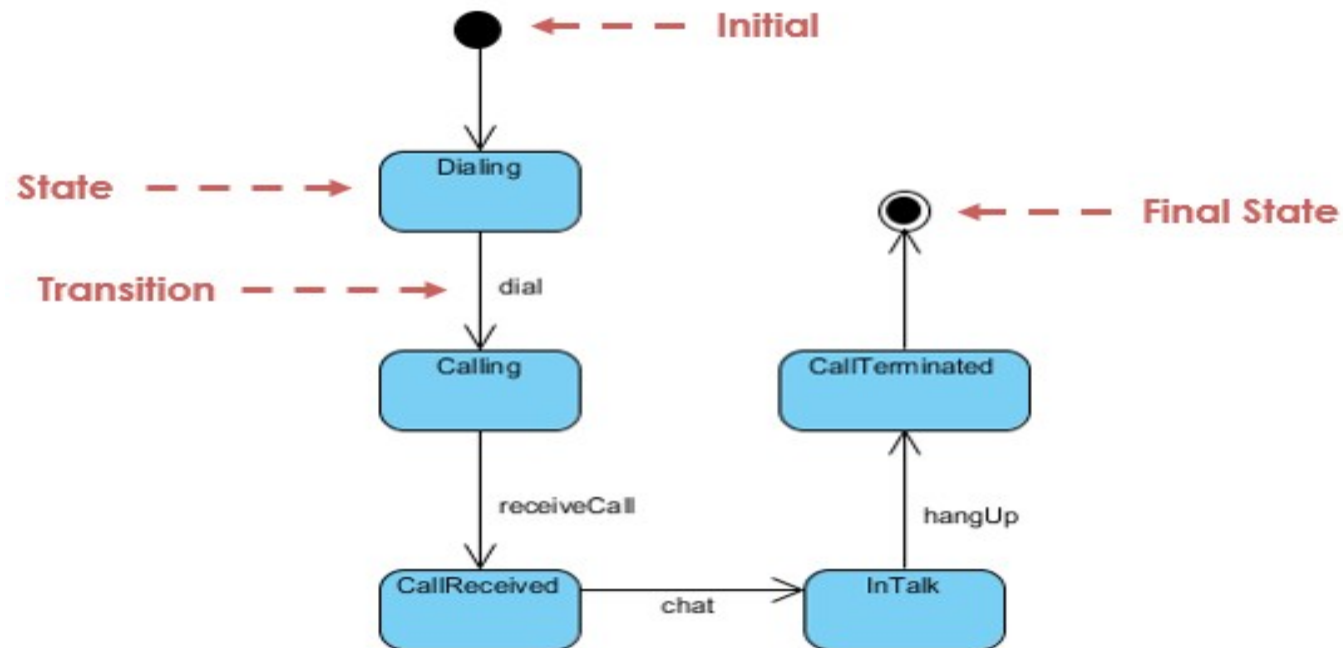
Activity Diagram Example



State Machine Diagram

- A state diagram is a type of diagram used in UML to describe the behavior of systems which is based on the concept of state diagrams by David Harel.
- State diagrams depict the permitted states and transitions as well as the events that effect these transitions.
- It helps to visualize the entire lifecycle of objects and thus help to provide a better understanding of state-based systems.

State Machine Diagram Example

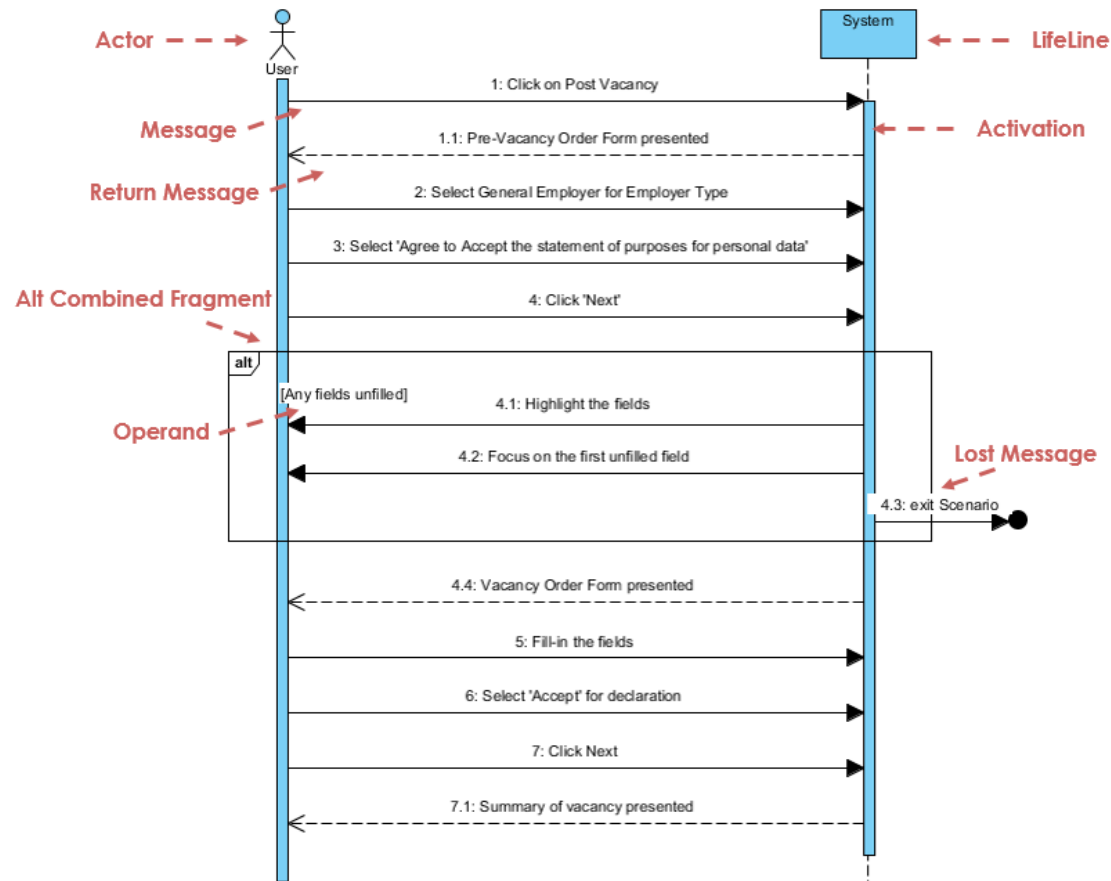


Sequence Diagram

- The Sequence Diagram models the collaboration of objects based on a time sequence.
- It shows how the objects interact with others in a particular scenario of a use case.
- With the advanced visual modeling capability, you can create complex sequence diagram in few clicks.
- Besides, some modeling tool such as Visual Paradigm can generate sequence diagram from the flow of events which you have defined in the use case description.

Cont'd

Sequence Diagram Example

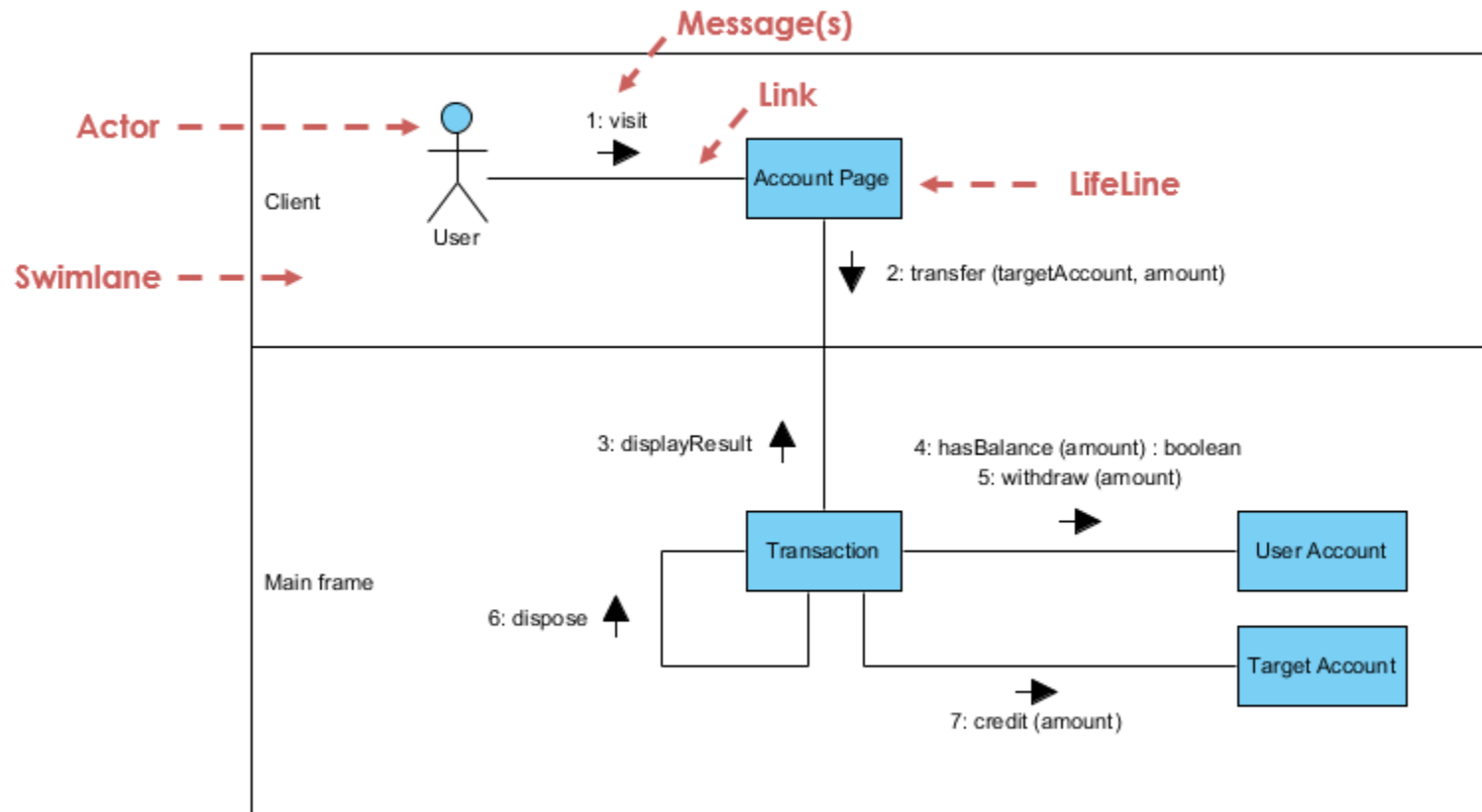


Communication Diagram

- Similar to Sequence Diagram, the Communication Diagram is also used to model the dynamic behavior of the use case.
- When compare to Sequence Diagram, the Communication Diagram is more focused on showing the collaboration of objects rather than the time sequence.
- They are actually semantically equivalent, so some of the modeling tool such as, Visual Paradigm allows you to generate it from one to the other.

Cont'd

Communication Diagram Example

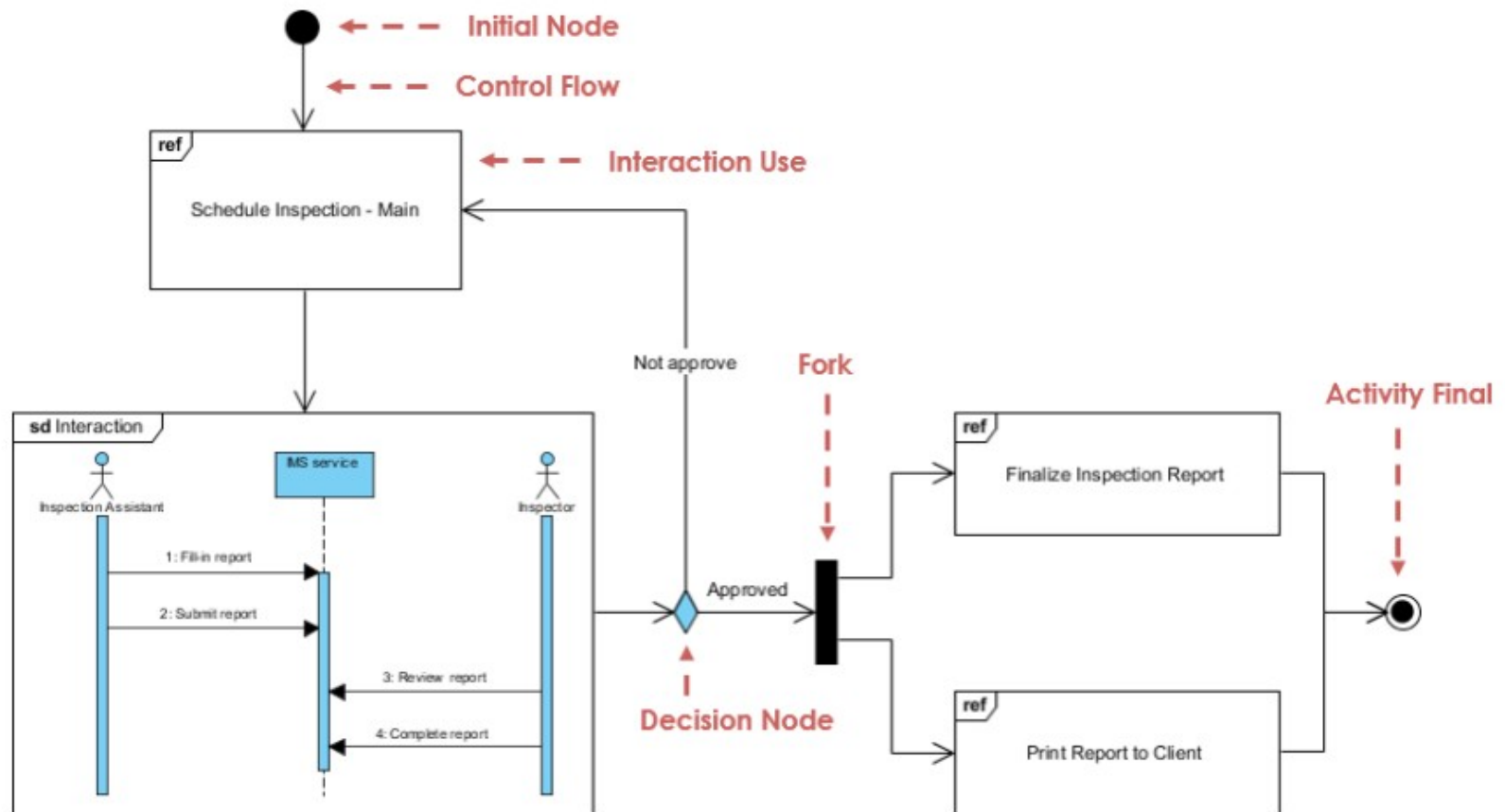


Interaction Overview Diagram

- The Interaction Overview Diagram focuses on the overview of the flow of control of the interactions.
- It is a variant of the Activity Diagram where the nodes are the interactions or interaction occurrences.
- The Interaction Overview Diagram describes the interactions where messages and lifelines are hidden. You can link up the "real" diagrams and achieve high degree navigability between diagrams inside the Interaction Overview Diagram.

Cont'd

Interaction Overview Diagram Example

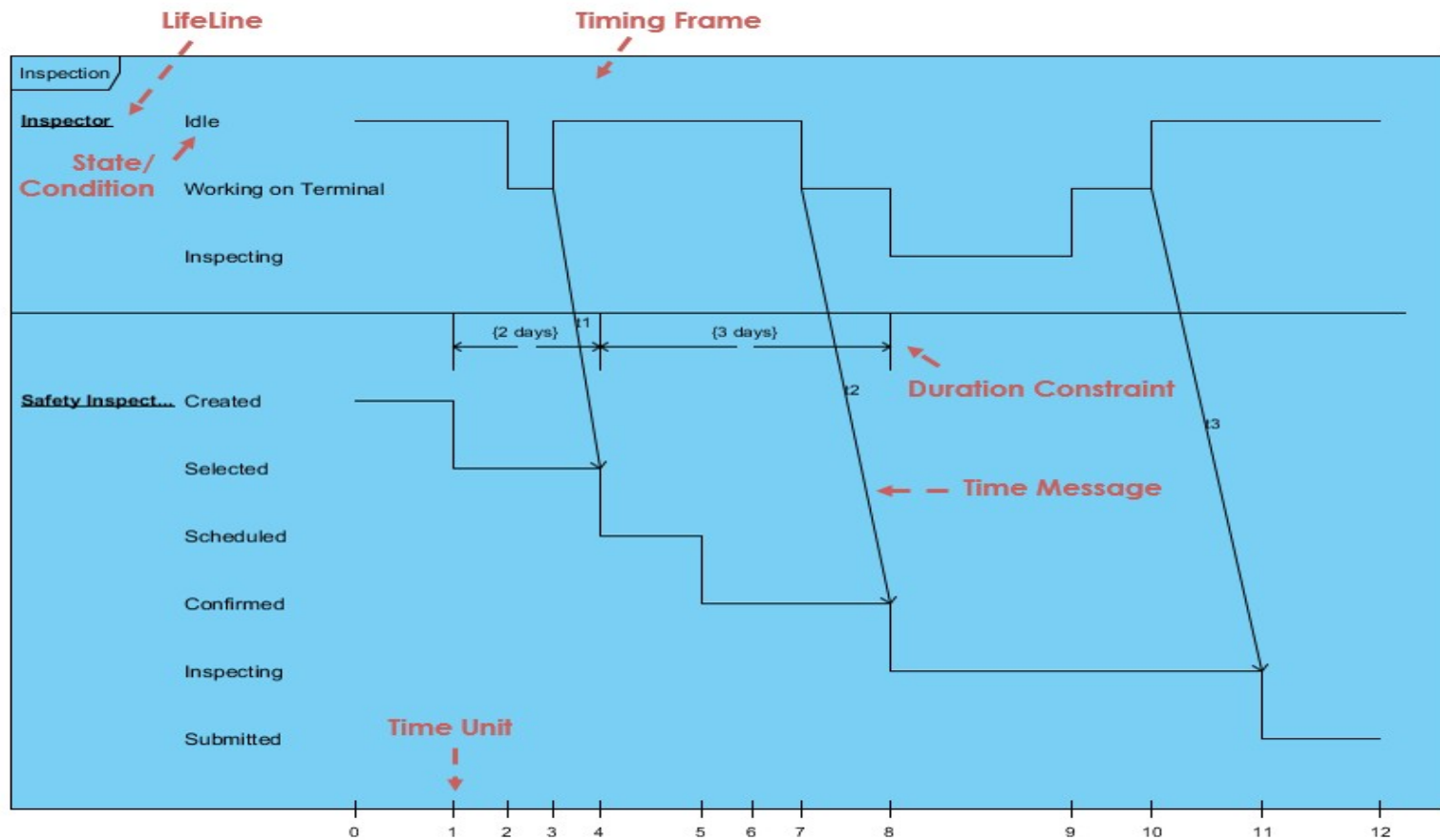


Timing Diagram

- Timing Diagram shows the behavior of the object(s) in a given period of time.
- Timing diagram is a special form of a sequence diagram.
- The differences between timing diagram and sequence diagram are the axes are reversed so that the time are increase from left to right and the lifelines are shown in separate compartments arranged vertically.

Cont'd

Timing Diagram Example



Thank you!